Section A: Very Short Answer Questions

1. What is a data frame in R?
Answer: A data frame is a two-dimensional table that stores data in rows and columns.

2. Which function is used to create a data frame in R?
Answer: data.frame()

3. How do you check the structure of a data frame?
Answer: str()

4. Write the command to display the first six rows of a data frame.
Answer: head()

5. How do you find the number of rows in a data frame?
Answer: nrow()

Section B: Short Answer Questions

1. Explain how a data frame is different from a matrix in R.

Answer:
A data frame can store different data types in different columns, whereas a matrix can store only one data type.

2. How can you add a new column to an existing data frame?

Answer:
Use the $ operator to add a new column.

Example:

```
student$Age <- c(18, 19, 20)
```

3. Write R code to rename columns of a data frame.

Answer:
Use the colnames() function.

Example:

```
colnames(student) <- c("RollNo", "Name", "Marks")
```

4. How do you extract a specific column from a data frame?

Answer:
Use the $ operator or column indexing.

Example:

```
student$Marks
```

or

```
student[, "Marks"]
```

5. What is the use of the subset() function?

Answer:
The subset() function is used to select specific rows and columns from a data frame based on a condition.

Example:

```
subset(student, Marks > 80)
```

Section C: Long Answer / Descriptive Questions

1. Explain different methods to create a data frame in R with examples.

Answer:

A **data frame** is a two-dimensional data structure in R that stores data in rows and columns. Each column can have a different data type.

Method 1: Using data.frame()

The data.frame() function is the most common way to create a data frame.

Example:

```
student <- data.frame(
  RollNo = c(1, 2, 3),
  Name = c("Asha", "Rahul", "Priya"),
  Marks = c(85, 90, 88)
)
print(student)
```

Method 2: Creating an Empty Data Frame

An empty data frame can be created and values can be added later.

Example:

```
student <- data.frame()
student$RollNo <- c(1, 2, 3)
student$Name <- c("Asha", "Rahul", "Priya")
student$Marks <- c(85, 90, 88)
print(student)
```

Method 3: Converting Vectors into a Data Frame

Multiple vectors can be combined into a data frame.

Example:

```
roll <- c(1, 2, 3)
name <- c("Asha", "Rahul", "Priya")
marks <- c(85, 90, 88)

student <- data.frame(roll, name, marks)
print(student)
```

Method 4: Importing Data from a CSV File

A CSV file can be read and converted into a data frame.

Example:

```
student <- read.csv("student.csv")
print(student)
```

Conclusion

These methods are commonly used to create data frames in R. The data.frame() function is the simplest and most widely used method for storing tabular data.

2. Discuss various ways to access rows and columns in a data frame.

Answer:

In R, rows and columns of a data frame can be accessed in different ways.

1. Access a Single Column using $

The $ operator is used to access a specific column.

Example:

student$Marks

2. Access a Column using Column Name

Use square brackets with the column name.

Example:

student[, "Marks"]

3. Access a Row by Row Number

Specify the row number inside square brackets.

Example:

student[2,]

4. Access a Specific Cell

Specify both row and column numbers.

Example:

student[2, 3]

5. Access Multiple Rows and Columns

Specify the required row and column numbers.

Example:

student[1:2, 2:3]

6. Access Rows Based on a Condition

Use the subset() function or logical condition.

Example:

```
subset(student, Marks > 80)
```

or

```
student[student$Marks > 80, ]
```

Conclusion

R provides several methods to access data in a data frame, such as using the $ operator, square brackets ([]), and the subset() function. These methods make it easy to retrieve specific rows, columns, or individual values.

3. Explain how to add, delete, and rename columns in a data frame with examples.

Answer:

A data frame in R allows users to modify its columns by adding new columns, deleting existing columns, and renaming column names.

1. Adding a New Column

A new column can be added using the $ operator.

Example:

```
student$Age <- c(18, 19, 20)
print(student)
```

2. Deleting a Column

A column can be deleted by assigning NULL to it.

Example:

```
student$Age <- NULL
print(student)
```

3. Renaming Columns

The colnames() function is used to rename column names.

Example:

```
colnames(student) <- c("RollNo", "StudentName", "Marks")
print(student)
```

Output Example

RollNo	StudentName	Marks
1	Asha	85
2	Rahul	90
3	Priya	88

Conclusion

R provides simple functions to modify data frames. The $ operator is used to add or delete columns, while the colnames() function is used to rename columns, making data management easy and efficient.

Question 4:
Describe how to merge two data frames with examples.

Answer:

Merging data frames in R means combining two or more data frames into a single data frame based on one or more common columns. The merge() function is used for this purpose.

Syntax:

```
merge(x, y, by = "column_name")
```

Where:

- x = First data frame
- y = Second data frame
- by = Common column used to merge the data

Example:

```
# Create first data frame
student <- data.frame(
  RollNo = c(101, 102, 103),
  Name = c("Alice", "Bob", "Charlie")
)

# Create second data frame
marks <- data.frame(
  RollNo = c(101, 102, 103),
  Marks = c(85, 90, 88)
)

# Merge the data frames
result <- merge(student, marks, by = "RollNo")

# Display merged data
print(result)
```

Output:

```
  RollNo    Name Marks
1    101   Alice    85
2    102     Bob    90
3    103 Charlie    88
```

Explanation:

- Both data frames have a common column **RollNo**.
- The merge() function combines the rows with matching RollNo values.
- The resulting data frame contains columns from both data frames.

Advantages of Merging Data Frames

- Combines related data from multiple sources.
- Avoids duplicate information.
- Makes data analysis easier.
- Useful in data preprocessing and reporting.

Conclusion:
The merge() function is an efficient way to combine multiple data frames using a common key, making data management and analysis easier in R.

\Question 5:
Explain sorting and ordering of data frames in R.

Answer:

Sorting and ordering in R are used to arrange the rows of a data frame in ascending or descending order based on one or more columns. The order() function is commonly used for this purpose.

Syntax:

dataframe[order(dataframe$column_name),]

For descending order:

dataframe[order(-dataframe$column_name),]

Example:

```
# Create a data frame
student <- data.frame(
  RollNo = c(103, 101, 102),
  Name = c("Charlie", "Alice", "Bob"),
  Marks = c(88, 85, 90)
)

# Sort by RollNo (Ascending)
sorted_roll <- student[order(student$RollNo), ]
print(sorted_roll)
```

```
# Sort by Marks (Descending)
sorted_marks <- student[order(-student$Marks), ]
print(sorted_marks)
```

Output:

Sorted by RollNo (Ascending):

```
  RollNo    Name Marks
2    101   Alice    85
3    102     Bob    90
1    103 Charlie    88
```

Sorted by Marks (Descending):

```
  RollNo    Name Marks
3    102     Bob    90
1    103 Charlie    88
2    101   Alice    85
```

Explanation:

- order(student$RollNo) sorts the data frame in ascending order of Roll Number.
- order(-student$Marks) sorts the data frame in descending order of Marks.
- The order() function returns the row indices in the required order, and the data frame is rearranged accordingly.

Advantages of Sorting Data Frames

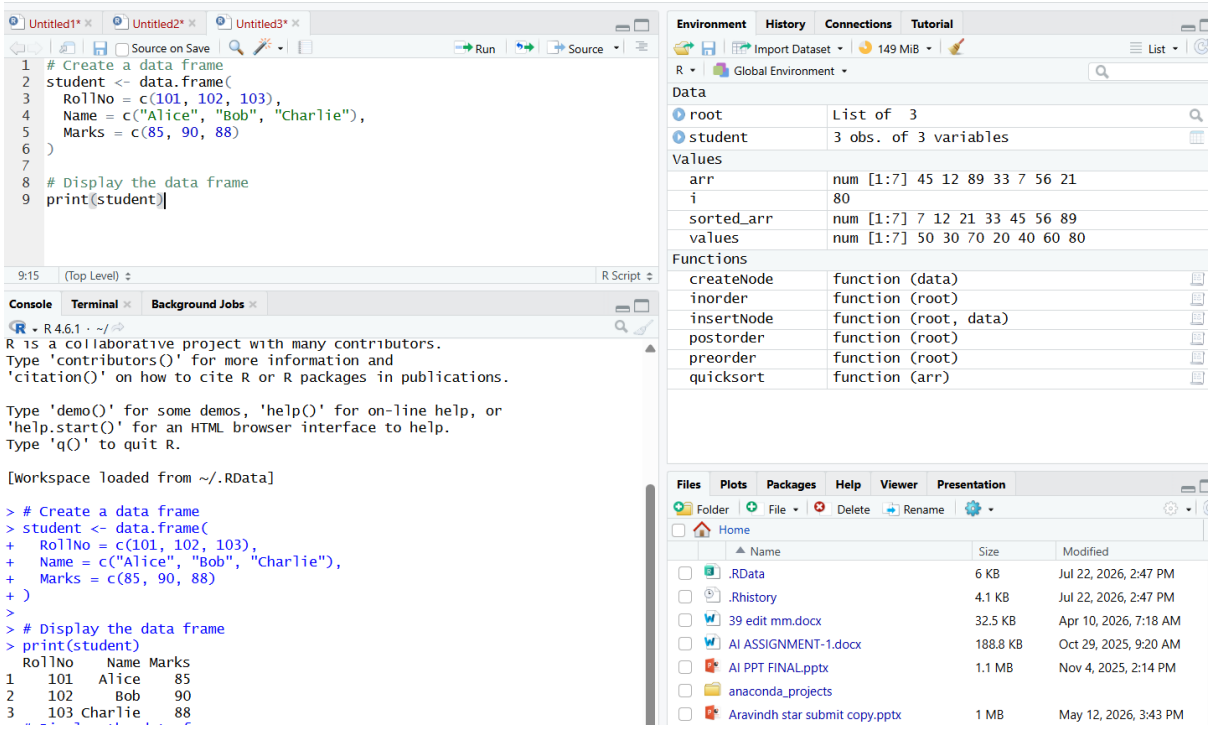
- Organizes data for easy analysis.
- Helps identify highest and lowest values.
- Makes searching and reporting easier.
- Useful for ranking and comparison.

Conclusion:

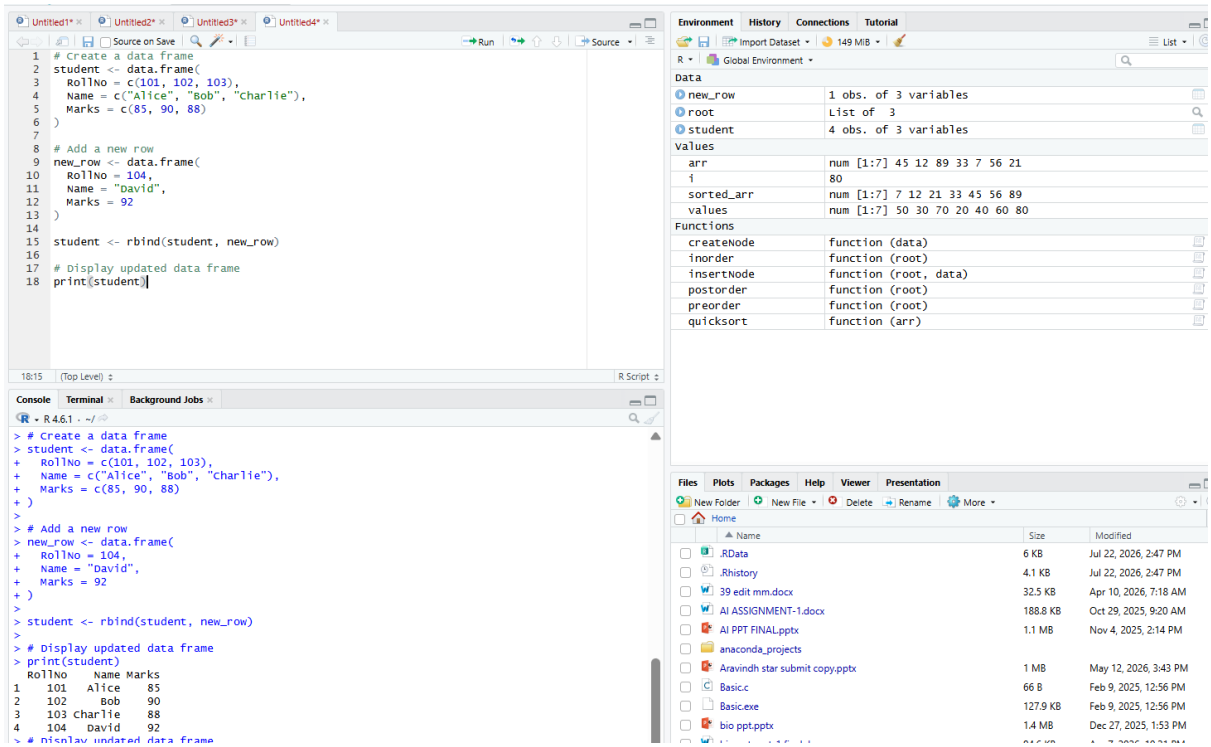
Sorting and ordering are important data manipulation techniques in R. The order() function allows users to arrange data frames efficiently in ascending or descending order based on one or more columns.

Section D: Practical / Programming Questions

Question 1: Create a data frame containing student details (Roll No, Name, Marks) and display it.



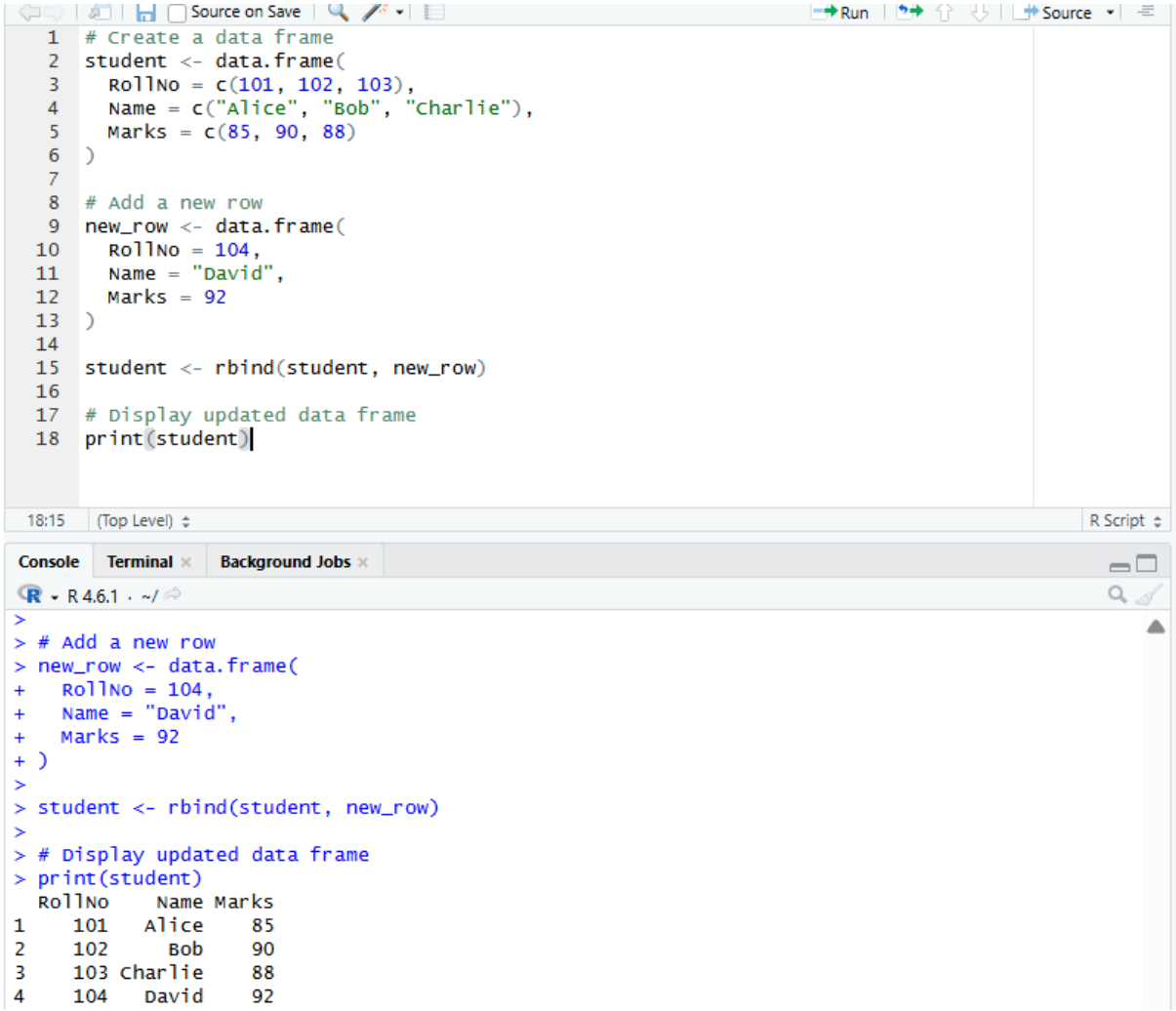
Question 2: Write an R program to add a new row to an existing data frame.



```
# Create a data frame
student <- data.frame(
  RollNo = c(101, 102, 103),
  Name = c("Alice", "Bob", "Charlie"),
  Marks = c(85, 90, 88)
)

# Add a new row
new_row <- data.frame(
  RollNo = 104,
  Name = "David",
  Marks = 92
)

student <- rbind(student, new_row)

# Display updated data frame
print(student)
```

```
> # Create a data frame
> student <- data.frame(
+   RollNo = c(101, 102, 103),
+   Name = c("Alice", "Bob", "Charlie"),
+   Marks = c(85, 90, 88)
+ )
>
> # Add a new row
> new_row <- data.frame(
+   RollNo = 104,
+   Name = "David",
+   Marks = 92
+ )
>
> student <- rbind(student, new_row)
>
> # Display updated data frame
> print(student)
  RollNo    Name Marks
1    101   Alice    85
2    102     Bob    90
3    103 Charlie    88
4    104   David    92
```

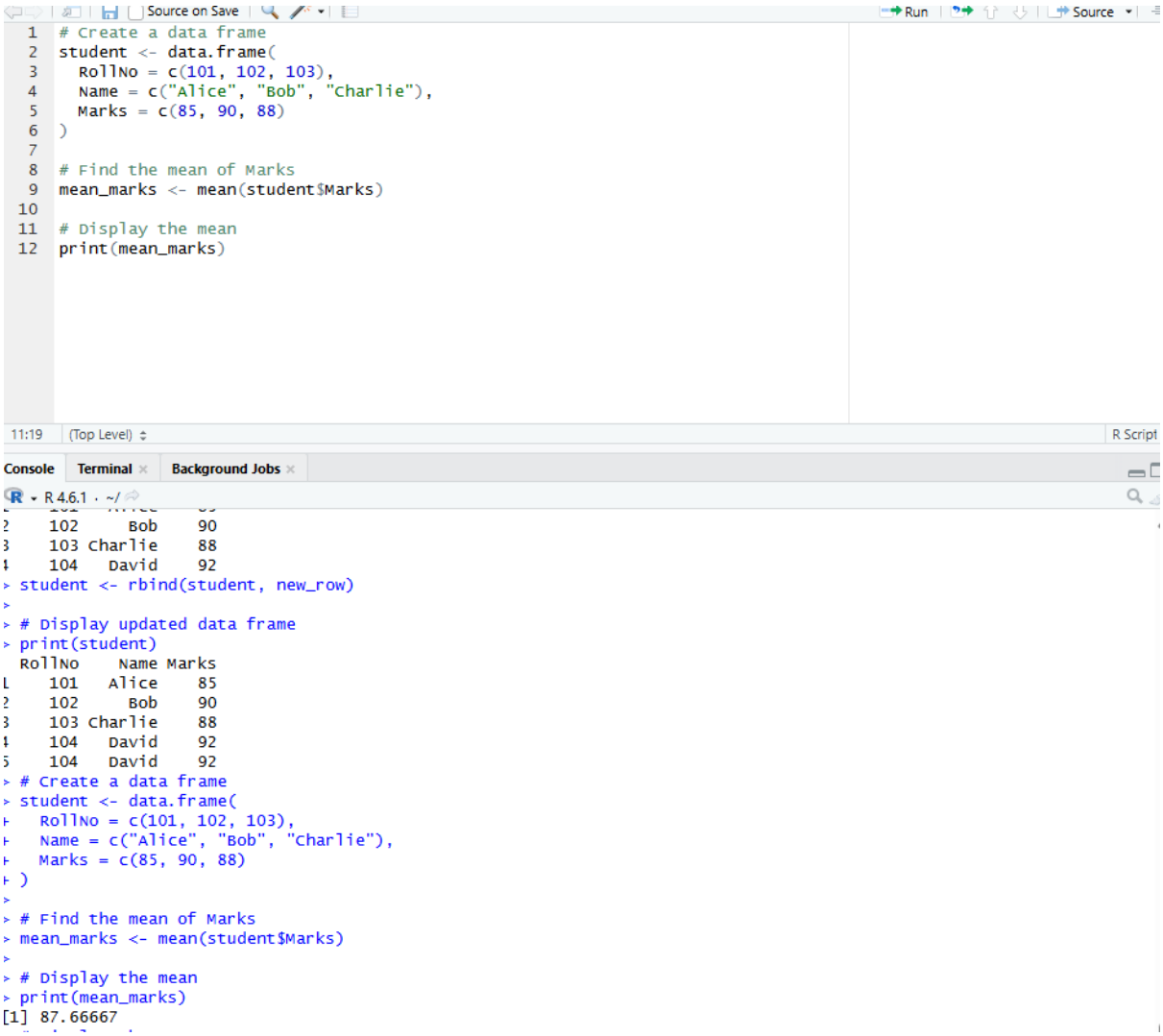
Question 3: Write R code to remove a column from a data frame.



```r
# Create a data frame
student <- data.frame(
  RollNo = c(101, 102, 103),
  Name = c("Alice", "Bob", "Charlie"),
  Marks = c(85, 90, 88)
)

# Add a new row
new_row <- data.frame(
  RollNo = 104,
  Name = "David",
  Marks = 92
)

student <- rbind(student, new_row)

# Display updated data frame
print(student)
```

```
> # Add a new row
> new_row <- data.frame(
+   RollNo = 104,
+   Name = "David",
+   Marks = 92
+ )
>
> student <- rbind(student, new_row)
>
> # Display updated data frame
> print(student)
  RollNo    Name Marks
1    101   Alice    85
2    102     Bob    90
3    103 Charlie    88
4    104   David    92
```

Question 4: Create a data frame and find the mean of a numeric column.



```
# Create a data frame
student <- data.frame(
  RollNo = c(101, 102, 103),
  Name = c("Alice", "Bob", "Charlie"),
  Marks = c(85, 90, 88)
)

# Find the mean of Marks
mean_marks <- mean(student$Marks)

# Display the mean
print(mean_marks)
```

```
2    102     Bob    90
3    103 Charlie    88
4    104   David    92
> student <- rbind(student, new_row)
>
> # Display updated data frame
> print(student)
  RollNo    Name Marks
1    101   Alice    85
2    102     Bob    90
3    103 Charlie    88
4    104   David    92
5    104   David    92
> # Create a data frame
> student <- data.frame(
+   RollNo = c(101, 102, 103),
+   Name = c("Alice", "Bob", "Charlie"),
+   Marks = c(85, 90, 88)
+ )
>
> # Find the mean of Marks
> mean_marks <- mean(student$Marks)
>
> # Display the mean
> print(mean_marks)
[1] 87.66667
```

Question 5: Write a program to filter rows based on a condition.

Answer:

```r
# Create a data frame
student <- data.frame(
  RollNo = c(101, 102, 103, 104),
  Name = c("Alice", "Bob", "Charlie", "David"),
  Marks = c(85, 90, 88, 75)
)

# Filter students with Marks greater than 85
result <- subset(student, Marks > 85)

# Display filtered rows
print(result)
```

```
> # Display the mean
> print(mean_marks)
[1] 87.66667
> # Display the mean
> print(mean_marks)
[1] 87.66667
> # Find the mean of Marks
> mean_marks <- mean(student$Marks)
>
> # Display the mean
> print(mean_marks)
[1] 87.66667
> # Create a data frame
> student <- data.frame(
+   RollNo = c(101, 102, 103, 104),
+   Name = c("Alice", "Bob", "Charlie", "David"),
+   Marks = c(85, 90, 88, 75)
+ )
>
> # Filter students with Marks greater than 85
> result <- subset(student, Marks > 85)
>
> # Display filtered rows
> print(result)
  RollNo    Name Marks
2    102     Bob    90
3    103 Charlie    88
```